

Uploading the Model Files

1) How File Management Works on the Hub

The Hugging Face Hub relies on two technologies to manage files:

File Type	System Used	Purpose
Regular files (<code>config.json</code> , <code>README.md</code> , small scripts)	<code>git</code>	Version control
Large files (model weights, checkpoints)	<code>git-lfs</code>	Large File Storage

Why Are `git` and `git-lfs` Separate?

- Standard `git` is not efficient for handling very large files.
 - Model weight files are often hundreds of MBs or even several GBs.
 - `git-lfs` stores large file data separately while keeping lightweight references in the repository.
 - This design primarily solves scalability and performance issues.
-

2) Methods for Uploading Files to the Hub

There are three main approaches:

1. Using the `huggingface_hub` Python library
2. Using direct `git` commands
3. Using the `upload_file` method

All three achieve the same goal (uploading files), but the workflow and level of control differ.

1) Setup (Install)

```
pip install -U huggingface_hub
```

2) Login (Using a Token)

Option A — CLI (Recommended)

```
huggingface-cli login
```

Paste your access token when prompted.

Option B — From Python

```
from huggingface_hub import login
```

```
login("hf_your_token_here") # Direct token usage
```

Security Tip: Avoid hardcoding your token in source code. Use environment variables instead.

3) Create Repository + Upload Entire Folder

Assume your local model directory is:

```
./my_model/
```

It typically contains:

- `config.json`
 - Tokenizer files (`tokenizer.json`, `vocab.txt`, etc.)
 - `pytorch_model.bin` or `model.safetensors`
 - `README.md`
-

Example: Create Repository and Upload Entire Folder

```
import os
from huggingface_hub import HfApi, create_repo, upload_folder

# ---- Configuration ----
repo_id = "YOUR_USERNAME/my-awesome-model" # Format: "<user_or_org>/<repo_name>"
local_folder = "./my_model"

commit_message = "Initial upload: config + tokenizer + weights + README"

# ---- Optional: Load token from environment variable ----
# export HF_TOKEN=hf_xxx
token = os.environ.get("HF_TOKEN")

api = HfApi(token=token)

# ---- Create repository (ignore if already exists) ----
create_repo(
    repo_id=repo_id,
    private=False, # Set True if you want a private repo
    exist_ok=True,
    token=token
)

# ---- Upload entire folder ----
upload_folder(
    repo_id=repo_id,
    folder_path=local_folder,
    path_in_repo="", # Upload to repo root
    commit_message=commit_message,
    token=token
)

print("Done! Uploaded to:", repo_id)
```

4) Single File Upload (**upload_file**)

Example

If you only want to update `README.md`:

```
import os
from huggingface_hub import upload_file

repo_id = "YOUR_USERNAME/my-awesome-model"
token = os.environ.get("HF_TOKEN")

upload_file(
    path_or_fileobj="./my_model/README.md",
    path_in_repo="README.md",
    repo_id=repo_id,
    commit_message="Update README (model card)",
    token=token
)

print("README updated.")
```

5) Transformers Workflow: `save_pretrained` → `push_to_hub` (Most Seamless)

If you already have `model` and `tokenizer` objects:

```
import os
from transformers import AutoModelForSequenceClassification, AutoTokenizer

repo_id = "YOUR_USERNAME/my-awesome-model"
token = os.environ.get("HF_TOKEN")

model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

# Save locally
save_dir = "./my_model"
model.save_pretrained(save_dir)
tokenizer.save_pretrained(save_dir)
```

```
# Push to Hub
model.push_to_hub(repo_id, token=token, commit_message="Upload model weights")
tokenizer.push_to_hub(repo_id, token=token, commit_message="Upload tokenizer files")

print("Pushed model + tokenizer to:", repo_id)
```

2) Git + Git LFS Approach

Concept

The Hugging Face Hub is essentially a **git remote repository**.
So you follow the standard git workflow:

- `git clone`
- add files (`git add`)
- `git commit`
- `git push`

For large files (e.g., model weights), you use **Git LFS** (`git-lfs`) to track and push them properly.

0) Prerequisites (One-time Setup)

Check if Git is installed

```
git --version
```

Install + Enable Git LFS

Linux (Ubuntu/Debian):

```
sudo apt-get update  
sudo apt-get install -y git-lfs  
git lfs install
```

macOS (Homebrew):

```
brew install git-lfs  
git lfs install
```

Windows:

Install Git LFS (via Git for Windows + Git LFS installer), then run:

```
git lfs install
```

1) Create a Repository on the Hub

Create a new repository from the Web UI at huggingface.co/new (or via CLI/API as well).

Example repo name:

USERNAME/my-awesome-model

2) Clone the Repository Locally

```
git clone https://huggingface.co/USERNAME/my-awesome-model
cd my-awesome-model
```

If it's a private repository, you'll need authentication (see below).

3) Copy Your Model Files Into the Repo

Assume your trained model is in: `./my_model/`

Linux/macOS:

```
cp -r ./my_model/* .
```

Windows (PowerShell):

```
Copy-Item -Recurse ..\my_model\* .
```

4) Track Large Files with Git LFS

Large files commonly include:

- `pytorch_model.bin`

- `model.safetensors`
- `*.bin`, `*.safetensors`, `*.pt`, `*.ckpt`, `*.onnx`, etc.

Track them using:

```
git lfs track "*.bin"
git lfs track "*.safetensors"
git lfs track "*.pt"
git lfs track "*.ckpt"
```

This creates/updates a `.gitattributes` file — **you must commit it**.

Verify tracking

```
cat .gitattributes
git lfs ls-files
```

5) Add → Commit → Push

```
git add .
git commit -m "Add model files: config + tokenizer + weights + README"
git push
```

At this point, your files should be uploaded to the Hub.

Authentication (Pushing with a Token)

If `git push` asks for username/password or fails with auth errors, use a **Hugging Face token**.

Option A: CLI Login (Recommended)

```
huggingface-cli login
```

Paste your token, then push again:

```
git push
```


Option B: Put Token in Remote URL (Works but Not Recommended)

```
git remote set-url origin
```

```
https://USERNAME:hf_your_token@huggingface.co/USERNAME/my-awesome-model
```

```
git push
```

Security note: This can leave your token in shell history, so CLI login is safer.

3) `upload_file` Approach (`huggingface_hub`)

Concept

`upload_file` is a function from the `huggingface_hub` library that lets you:

- Upload a **single specific file** to a Hugging Face Hub repository
- **Without cloning the repo**
- **Without running git commands**
- While also providing a **commit message**

It's basically like doing “**one-file commit via an API call.**”

When to Use It

`upload_file` is best when:

- You only need to update **one or two files** (e.g., `README.md`, `config.json`)
- You want to push a **weights file** from a training pipeline after training finishes
- You need a quick update from **Notebook/Colab**
- You want to avoid the full `git-lfs/clone/commit` workflow

If you are uploading a whole model folder or many files, `upload_folder` is usually more convenient.

How It Works (Conceptually)

You specify:

- Local file path: `path_or_fileobj`
- Destination path inside the repo: `path_in_repo`
- Target repo: `repo_id`
- Commit message: `commit_message`

Then the Hub creates a commit containing that file.

Minimal Example: Upload a README

```
import os
from huggingface_hub import upload_file

token = os.environ["HF_TOKEN"] # set env first: export HF_TOKEN=hf_xxx
repo_id = "YOUR_USERNAME/my-awesome-model"

upload_file(
    path_or_fileobj="./README.md",    # local file
    path_in_repo="README.md",        # destination path in the repo
    repo_id=repo_id,
    commit_message="Add/Update README (model card)",
    token=token
)

print("README uploaded.")
```

Example: Upload Model Weights (.safetensors / .bin)

```
import os
from huggingface_hub import upload_file

token = os.environ["HF_TOKEN"]
repo_id = "YOUR_USERNAME/my-awesome-model"

upload_file(
    path_or_fileobj="./my_model/model.safetensors",
    path_in_repo="model.safetensors",
    repo_id=repo_id,
    commit_message="Upload model weights (safetensors)",
    token=token
)

print("Weights uploaded.")
```

If your file is `pytorch_model.bin`, then use:

```
path_in_repo="pytorch_model.bin"
```

Example: Upload Into a Subfolder (e.g., **checkpoints/**)

If you want to store files inside a folder like **checkpoints/**:

```
import os
from huggingface_hub import upload_file

token = os.environ["HF_TOKEN"]
repo_id = "YOUR_USERNAME/my-awesome-model"

upload_file(
    path_or_fileobj="./outputs/ckpt-1000/model.safetensors",
    path_in_repo="checkpoints/ckpt-1000/model.safetensors",
    repo_id=repo_id,
    commit_message="Add checkpoint ckpt-1000",
    token=token
)

print("Checkpoint uploaded.")
```